

Strings and String Manipulation in Python

1. Introduction to Strings

A **string** is a sequence of characters enclosed within single quotes (' '), double quotes (" "), or triple quotes (''' ' or "" "" """).

Strings are one of the most commonly used data types in Python. They are used to store text such as names, addresses, messages, passwords, and more.

Examples

```
name = "John"
city = 'Mumbai'
message = """Welcome to Python Programming"""
```

Output

```
print(name)
print(city)
print(message)
```

2. Characteristics of Strings

Strings are Ordered

Each character has a position called an **index**.

```
text = "Python"
```

Character P y t h o n

Index	0	1	2	3	4	5
-------	---	---	---	---	---	---

Strings are Immutable

Once a string is created, its characters cannot be changed directly.

 Incorrect:

```
name = "Python"
name[0] = "J"
```

This will generate an error.

 Correct:

```
name = "Python"
name = "Jython"
print(name)
```

Output:

Jython

3. Creating Strings

Single Quotes

```
name = 'Rahul'
```

Double Quotes

```
name = "Rahul"
```

Triple Quotes

Used for multi-line strings.

```
message = """  
Hello Students,  
Welcome to Python Programming.  
"""  
print(message)
```

4. Accessing Characters in a String

Positive Indexing

```
text = "Python"
```

```
print(text[0])  
print(text[1])  
print(text[5])
```

Output:

```
P  
y  
n
```

Negative Indexing

Python allows accessing characters from the end.

```
text = "Python"
```

```
print(text[-1])
```

```
print(text[-2])  
print(text[-6])
```

Output:

```
n  
o  
p
```

5. String Slicing

Slicing allows extracting a portion of a string.

Syntax

```
string[start:end]
```

The end index is excluded.

Example

```
text = "PythonProgramming"
```

```
print(text[0:6])  
print(text[6:17])
```

Output:

```
Python  
Programming
```

Omitting Start Index

```
text = "Python"
```

```
print(text[:3])
```

Output:

```
Pyt
```

Omitting End Index

```
text = "Python"
```

```
print(text[2:])
```

Output:

thon

Using Step Value

```
text = "Python"
```

```
print(text[0:6:2])
```

Output:

Pto

Reverse a String

```
text = "Python"
```

```
print(text[::-1])
```

Output:

nohtyP

6. String Length

The len() function returns the number of characters.

```
text = "Python"
```

```
print(len(text))
```

Output:

6

7. String Concatenation

Joining two or more strings.

```
first_name = "John"
```

```
last_name = "Doe"
```

```
full_name = first_name + " " + last_name
```

```
print(full_name)
```

Output:

John Doe

8. String Repetition

Using * operator.

```
text = "Python "
```

```
print(text * 3)
```

Output:

Python Python Python

9. Checking Membership

Using in and not in.

```
text = "Python Programming"
```

```
print("Python" in text)
```

```
print("Java" in text)
```

Output:

True

False

10. Common String Methods

String methods help manipulate and process text.

upper()

Converts all characters to uppercase.

```
text = "python"
```

```
print(text.upper())
```

Output:

PYTHON

lower()

Converts all characters to lowercase.

```
text = "PYTHON"
```

```
print(text.lower())
```

Output:

```
python
```

title()

Converts first letter of each word to uppercase.

```
text = "python programming"
```

```
print(text.title())
```

Output:

```
Python Programming
```

capitalize()

Capitalizes only the first character.

```
text = "python programming"
```

```
print(text.capitalize())
```

Output:

```
Python programming
```

swapcase()

Converts uppercase to lowercase and vice versa.

```
text = "PyThOn"
```

```
print(text.swapcase())
```

Output:

```
pYtHoN
```

11. Removing Extra Spaces

strip()

Removes spaces from both sides.

```
text = " Python "
```

```
print(text.strip())
```

Output:

Python

lstrip()

Removes spaces from left side.

```
text = " Python"
```

```
print(text.lstrip())
```

Output:

Python

rstrip()

Removes spaces from right side.

```
text = "Python "
```

```
print(text.rstrip())
```

Output:

Python

12. Searching in Strings

find()

Returns index of first occurrence.

```
text = "Python Programming"
```

```
print(text.find("Program"))
```

Output:

7

index()

Similar to find but gives error if not found.

```
text = "Python"
```

```
print(text.index("t"))
```

Output:

2

count()

Counts occurrences.

```
text = "banana"
```

```
print(text.count("a"))
```

Output:

3

13. Replacing Text

replace()

```
text = "I like Java"
```

```
new_text = text.replace("Java", "Python")
```

```
print(new_text)
```

Output:

I like Python

14. Splitting Strings

split()

Converts string into a list.

```
text = "Python Java C++"
```

```
languages = text.split()
```

```
print(languages)
```

Output:

```
['Python', 'Java', 'C++']
```

Split by Comma

```
data = "Apple,Banana,Mango"
```

```
print(data.split(","))
```

Output:

```
['Apple', 'Banana', 'Mango']
```

15. Joining Strings

join()

Combines list elements into a string.

```
languages = ["Python", "Java", "C++"]
```

```
result = ", ".join(languages)
```

```
print(result)
```

Output:

```
Python, Java, C++
```

16. String Validation Methods

isalpha()

Checks if all characters are alphabets.

```
name = "Python"
```

```
print(name.isalpha())
```

Output:

```
True
```

isdigit()

Checks if all characters are digits.

```
num = "12345"
```

```
print(num.isdigit())
```

Output:

True

isalnum()

Checks if string contains letters and numbers only.

```
text = "Python123"
```

```
print(text.isalnum())
```

Output:

True

islower()

```
text = "python"
```

```
print(text.islower())
```

Output:

True

isupper()

```
text = "PYTHON"
```

```
print(text.isupper())
```

Output:

True

17. String Formatting

Using Concatenation

```
name = "John"
age = 25

print("My name is " + name)
```

Using format()

```
name = "John"
age = 25

print("My name is {} and I am {} years old".format(name, age))
```

Output:

My name is John and I am 25 years old

Using f-Strings (Recommended)

```
name = "John"
age = 25

print(f"My name is {name} and I am {age} years old")
```

Output:

My name is John and I am 25 years old

18. Escape Characters

Used for special formatting.

Escape Character Meaning

<code>\n</code>	New Line
<code>\t</code>	Tab Space
<code>\\</code>	Backslash
<code>\'</code>	Single Quote
<code>\"</code>	Double Quote

Example

```
print("Hello\nWorld")
```

Output:

Hello
World

19. Practical Programs

Program 1: Reverse a String

```
text = input("Enter a string: ")

print("Reversed String:", text[::-1])
```

Program 2: Count Vowels

```
text = input("Enter a string: ")

count = 0

for ch in text.lower():
    if ch in "aeiou":
        count += 1

print("Total vowels:", count)
```

Program 3: Check Palindrome

```
text = input("Enter a string: ")

if text == text[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

Program 4: Count Words

```
sentence = input("Enter a sentence: ")

words = sentence.split()

print("Total words:", len(words))
```

Program 5: Password Validation

```
password = input("Enter password: ")

if len(password) >= 8:
    print("Valid Password")
else:
    print("Password must contain at least 8 characters")
```

20. Real-Life Applications of Strings

Student Management System

```
student_name = "Rahul Sharma"
```

Login System

```
username = "admin"
password = "1234"
```

Email Processing

```
email = "student@gmail.com"
```

Search Functionality

```
search = "Python"
```

Summary

Strings are used to store and manipulate text data in Python. Important concepts include:

- Creating strings
- Indexing and slicing
- String concatenation
- String repetition
- String methods
- String validation
- String formatting

- Splitting and joining strings
- Searching and replacing text
- Practical string-based programs

Mastering strings is essential because almost every Python application—from login systems to web development and data processing—uses string manipulation extensively.